

Document Summary This note explores techniques for efficient fine-tuning and inference of **LLMs**, addressing the computational challenges of traditional methods. It details **PEFT** approaches like **BitFit**, **Adapter Layers**, **LoRA**, and **Prompt Tuning**, alongside optimization strategies such as **Quantization**, reduced floating-point precision, and **Model Distillation**. The goal is to maximize LLM performance on specific tasks while minimizing resource consumption.

Efficient Fine-tuning and Inference for Large Language Models (LLMs)

The Need for Fine-tuning LLMs

While **Large Language Models (LLMs)** like **GPT-3** and **FLAN** demonstrate impressive few-shot or zero-shot learning capabilities, **fine-tuned** models consistently achieve the **upper bound in performance** for specific tasks. This often means they can outperform even larger models that rely solely on **In-Context Learning (ICL)**. Therefore, **fine-tuning** is a crucial step for maximizing the performance of an **LLM** on a particular application.

Fine-tuning

Fine-tuning This process improves a model's performance on new, specific tasks by continuing its training on data tailored to that task, building upon the knowledge acquired during its initial pretraining.

- During **fine-tuning**, all model weights can change, allowing for extensive adaptation.
- **Pros:**
 - It achieves performance levels comparable to training a model from scratch, but with significantly less data and computational cost.
 - It enables fundamental behavior changes, such as **instruction following (instruction tuning)** or aligning the model with human values (**model alignment**), even with relatively smaller datasets.
- **Cons:**

- It is very resource intensive, especially for large models, because a vast number of parameters need to be updated during the training process.

Feature-based Transfer

***Feature-based transfer** In this approach, the **backbone model** (the pretrained **LLM**) weights are **frozen**. This means their values are fixed and do not change during training; no gradient updates are applied to them.*

- Instead, a new, small, **trainable head** (typically a few layers) is added on top of the **frozen** backbone. Only the weights of this new head are updated. Optionally, the last few layers of the backbone model can be **unfrozen** to allow for some deeper adaptation.
- The original, **frozen** model effectively acts as a **feature extractor**, providing high-level representations (features) to the newly added head, which then learns to map these features to the desired task output.
- **Pros:**
 - It is less resource intensive compared to full **fine-tuning** because only a small fraction of the total weights are updated.
 - It is particularly effective when the new task is similar to the tasks the original model was pretrained on.
- **Cons:**
 - It can lead to sub-optimal performance for complex or highly diverse tasks that require more profound changes within the model's internal representations, which freezing the backbone prevents.

Problems with "Classic" Approaches

The traditional methods of **fine-tuning** (full **fine-tuning**) and **feature-based transfer** present a trade-off: full **fine-tuning** offers high performance but demands significant resources, while **feature-based transfer** is resource-efficient but may compromise performance on complex tasks. With the increasing size of **LLMs** and the diversity of tasks they are applied to, relying solely on either of these classic approaches becomes challenging. This challenge directly motivates the development of **Parameter-efficient Fine-Tuning (PEFT)** techniques.

Parameter-efficient Fine-Tuning (PEFT)

Parameter-efficient Fine-Tuning (PEFT) These are a collection of techniques specifically designed to reduce the computational cost and memory footprint of **fine-tuning LLMs** by drastically decreasing the number of parameters that need to be updated.

- Key **PEFT** techniques include:
 - **BitFit**
 - **Adapter layers**
 - **LoRA** (Low Rank Adaptation)
 - **Prompt tuning**

Bias-terms Fine-tuning (BitFit)

BitFit This is a sparse **fine-tuning** method that modifies only the **bias terms** within the model, or a selected subset of them.

- **Bias terms** constitute a very small fraction of the total model weights. For instance, in a **BERT** model, they might represent only about 0.1% of all parameters.
- Research by Zaken et al. (2021) demonstrated that by tuning only these **bias terms**, **BitFit** can achieve performance levels comparable to full **fine-tuning**.
- Historically, **bias terms** were sometimes overlooked (e.g., they weren't explicitly highlighted in the original Transformer paper). Their significant role in models like **BERT**, when selectively tuned, might be considered a "fortunate mistake" in their initial design.

Adapters

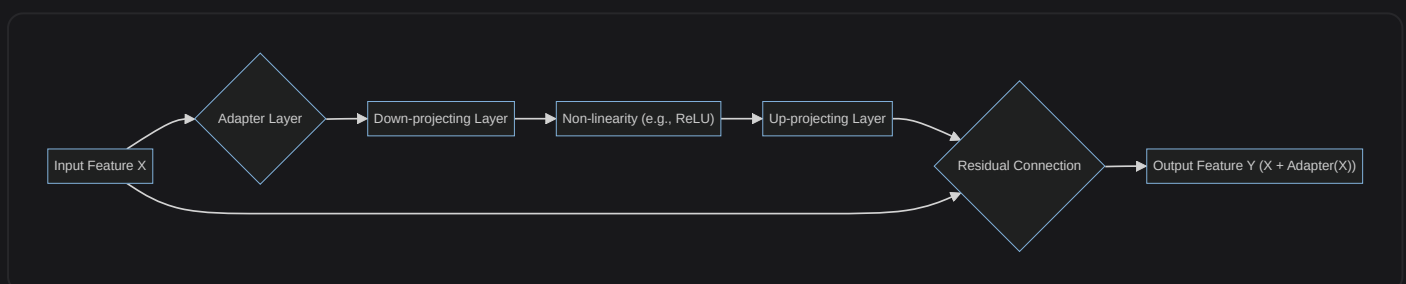
Adapters These are small, specialized layers that are strategically inserted between existing layers of a pretrained model. Common insertion points include between the attention layers or within the fully-connected layers of a Transformer block.

- During **fine-tuning**, only these **adapter layers** are trained; the vast majority of the pretrained model's weights remain **frozen**. The only other flexible components are typically the **Layer Norms**, which might be slightly updated.
- This approach significantly reduces the number of trained parameters while still allowing for deeper, more impactful changes to the model's internal representations compared to simple **feature-based transfer**.

Adapter Layer Architecture

Adapter layers are typically designed as simple, feed-forward (fully-connected) neural networks:

- A **down-projecting layer**: This layer reduces the dimensionality of the input.
 - Example: It might transform a 768-dimensional vector into a 32-dimensional vector.
- A **non-linearity**: An activation function like **ReLU** is applied to introduce non-linearity.
- An **up-projecting layer**: This layer restores the dimensionality back to the original size.
 - Example: It might transform the 32-dimensional vector back into a 768-dimensional vector.
- More complex architectures for **adapter layers** often do not yield significant additional benefits over this simple design.
- When an **adapter** is "injected" into a pretrained model, it is crucial that it initially acts as an **identity function**. This means $adapter(x) \approx x$ at the beginning of training. This prevents the **adapter** from "ruining" the finely tuned intermediate representations of the pretrained model. This **identity behavior** is achieved through a **residual connection** (where the original input x is added back to the **adapter's** output) and by initializing the weights of the **up-projecting layer** close to 0.



Adapter Layer Parameters Example: An **adapter layer** with a bottleneck dimension of $d_{bottleneck}$: Input dimension: d_{model} Output dimension: d_{model} Number of parameters in **down-projecting layer**: $d_{model} \times d_{bottleneck}$ Number of parameters in **up-projecting layer**: $d_{bottleneck} \times d_{model}$ Total parameters (excluding biases): $2 \times d_{model} \times d_{bottleneck}$

Results of Adapters

- **Parameter reduction**: Consider a single Transformer layer within a **BERT** model, which typically has approximately 7 million parameters. If we use **adapters** with a bottleneck dimension of 32, each **adapter layer** would have roughly 50 thousand parameters.

Adapter Parameter Calculation This is calculated as: $(768 \times 32) + 32$ (bias) + $(32 \times 768) + 768$ (bias) $\approx 50K$.

If two **adapters** are inserted per Transformer layer (e.g., one after attention, one after the feed-forward network), this amounts to about 100K parameters per layer. This represents a reduction by a factor of approximately 70x compared to the 7 million parameters of the full layer.

- **Performance:** Despite the significant reduction in trainable parameters, **adapters** have been shown to achieve performance levels similar to classic full **fine-tuning**, but at a mere fraction of the computational cost.

Low-Rank Adaptation (LoRA)

LoRA This technique freezes the original pretrained model's weight matrix W and instead only learns the **incremental change** ΔW that should be applied during **fine-tuning**. The updated weight matrix W' is then calculated as $W' = W + \Delta W$.

Rank of Matrices (Digression)

Rank of a matrix The **rank of a matrix** is defined as the maximum number of linearly independent rows (or columns) it contains.

- An $n \times n$ **full rank** matrix F has all its rows (and columns) linearly independent. Such a matrix can be factored into $F = AB^T$, where A and B are also $n \times n$ matrices.
- An $n \times n$ **low rank** matrix L has rows (and columns) that are linearly dependent. This means its rank r is less than n . A **low-rank** matrix can be factored into $L = AB^T$, where A and B are now $n \times r$ matrices, and r is the rank.
- **Example:** Consider the 3×3 matrix:

Example Low-Rank Matrix

$$\begin{pmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ -1 & -2 & -1 \end{pmatrix}$$

This matrix has a rank of 1. Notice that row 2 is simply row 1 multiplied by 2, and row 3 is row 1 multiplied by -1 . This matrix can be factored into:

Low-Rank Matrix Factorization

$$A = \begin{pmatrix} 1 \\ 2 \\ -1 \end{pmatrix} \quad \text{and} \quad B = \begin{pmatrix} 1 \\ 2 \\ 1 \end{pmatrix}$$

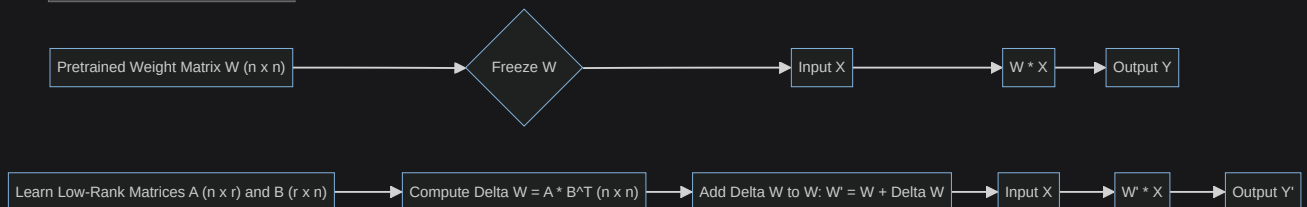
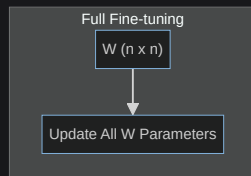
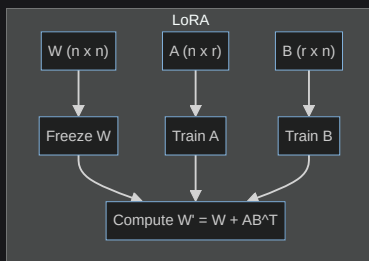
Then

$$AB^T = \begin{pmatrix} 1 \\ 2 \\ -1 \end{pmatrix} \begin{pmatrix} 1 & 2 & 1 \end{pmatrix} = \begin{pmatrix} 1 \cdot 1 & 1 \cdot 2 & 1 \cdot 1 \\ 2 \cdot 1 & 2 \cdot 2 & 2 \cdot 1 \\ -1 \cdot 1 & -1 \cdot 2 & -1 \cdot 1 \end{pmatrix} = \begin{pmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ -1 & -2 & -1 \end{pmatrix}$$

- **Near low-rank:** This term refers to matrices that are technically **full rank** but are very close to a **low-rank** matrix. Such matrices can be *approximated* effectively by a **low-rank** factorization AB^T . The quality of this approximation improves as the matrix's "distance" to a true **low-rank** matrix decreases.

LoRA (Low Rank Assumption of ΔW)

- Simply learning ΔW as a **full rank** matrix would not be efficient, as it would involve the same number of parameters as the original weight matrix W .
- **LoRA's** core insight is based on the empirical observation that the **incremental change** ΔW required during **fine-tuning** is generally **near low-rank**.
- Therefore, ΔW can be effectively approximated by factoring it into two much smaller matrices, A and B , such that $W' = W + AB^T$.
- During **fine-tuning**, we only learn the weights of these smaller matrices A and B , which collectively approximate ΔW with a chosen rank r .
- The parameter reduction is significant: If the original matrix W has dimensions $n \times n$, then ΔW also has $n \times n$ parameters. However, A has dimensions $n \times r$ and B has dimensions $r \times n$. The total parameters for A and B combined are $(n \times r) + (r \times n) = 2nr$. If we choose a rank r such that $r < n/2$, then $2nr < n^2$, meaning A and B collectively have fewer parameters than ΔW .
- **Initialization:** To ensure that the **incremental change** ΔW is initially zero (so $W' = W$ at the start of **fine-tuning**), matrix A is typically sampled from a normal distribution $\mathcal{N}(0, \sigma^2)$, while matrix B is initialized to all zeros. This makes $AB^T = 0$ at the beginning of training.



Results of LoRA

- Empirical evidence strongly supports the assumption that weight updates during **fine-tuning** are indeed **near low-rank**.
- A relatively small rank, such as $r = 4$, often yields excellent approximations for ΔW , demonstrating the effectiveness of the **low-rank** assumption.
- **Parameter reduction (BERT example)**: In a **BERT** Transformer layer, which has approximately 7 million parameters, applying **LoRA** with $r = 4$ to all four attention matrices (W_q, W_k, W_v, W_o , each 768×768) and both feed-forward (FF) layers ($768 \rightarrow 3072 \rightarrow 768$) results in a significantly reduced number of trainable parameters.

LoRA Parameter Calculation The calculation is approximately 50K parameters: $(768 \times 4 \times 2 \text{ (for A and B)} \times 4 \text{ (for } W_q, W_k, W_v, W_o)) + ((768 \times 4 + 3072 \times 4) \text{ for FF1} + (3072 \times 4 + 768 \times 4) \text{ for FF2})$.

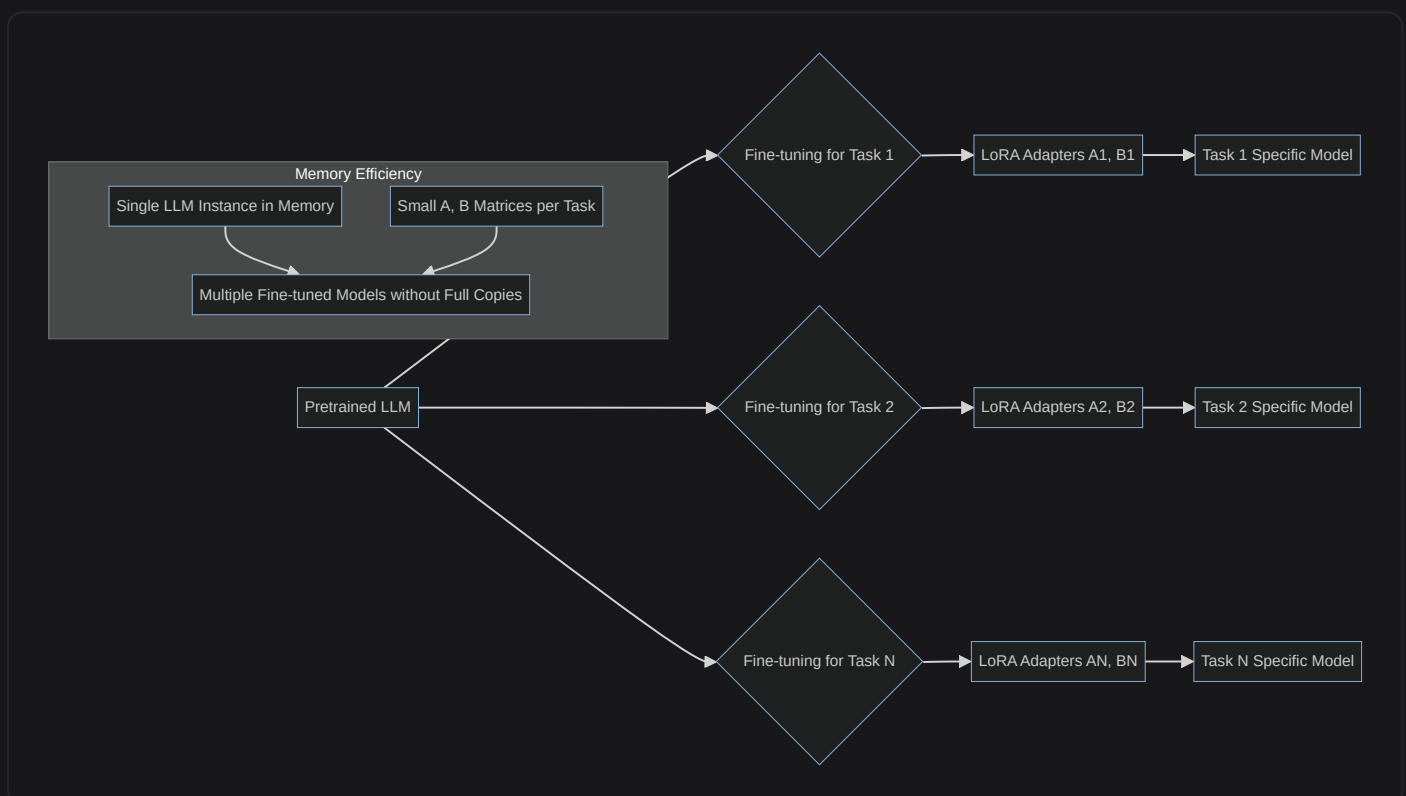
This is a reduction factor of approximately 140x fewer parameters compared to full **fine-tuning**.

- **LoRA** also allows for further parameter reduction by selectively applying it to only certain parts of the Transformer architecture, such as just the query and value matrices in the attention mechanism.

Considerations on LoRA

- **Scalability**: The benefits of **LoRA** become even more pronounced as model sizes increase. For extremely large models like **GPT-3** (with a model dimension $d_{model} = 12,288$), **LoRA** can lead to a reduction of trainable parameters by a factor of 10,000x or more.

- **Hardware:** While **LoRA** significantly reduces the number of *trainable* parameters, the original pretrained model weights still need to be loaded into memory. However, the memory required for storing gradients and optimizer states is drastically cut, leading to a typical reduction in hardware requirements (e.g., GPU memory) by up to 3x.
- **Multi-task learning:** **LoRA** is highly advantageous for scenarios involving multiple tasks. A single pretrained model instance can be loaded, and then **fine-tuned** for numerous distinct tasks. Each task would only require its own small set of A_t, B_t **adapter** matrices, making it extremely space-efficient for storing multiple **fine-tuned** models.
- **Inference time:** During inference, the updated weight matrix W' can be precomputed by adding W and AB^T together ($W' = W + AB^T$). This means that during actual inference, the operation $W'x$ becomes a single matrix multiplication, completely eliminating any overhead that might otherwise be introduced by the **LoRA** layers.



Prompt Tuning

Prompting: This technique involves adding extra input information to the model's original input, with the goal of conditioning the model to generate a desired output.

- Traditionally, **prompt design** involves carefully crafting and prepending specific vocabulary tokens (words or sub-word units) to the input. The aim is to maximize the likelihood of the model producing the correct or desired output.

Prompt tuning Instead of using existing vocabulary tokens, this method adds a fixed set of special, learnable tokens (often referred to as "soft prompts" or "virtual tokens") to the input prompt.

- During **fine-tuning**, only the embeddings of these special tokens are updated. The rest of the LLM's parameters remain **frozen**. This effectively "creates" optimal prompt words in the embedding space that guide the model towards the desired behavior for a specific task.

Other Optimization Techniques

Beyond **PEFT**, there are several other techniques aimed at reducing the memory footprint and computational cost of **LLMs**, both during training and inference. These methods often involve shrinking existing models or developing smaller, more efficient versions.

Quantization

Quantization This process involves mapping continuous (floating-point) values, which are typically used for model weights and activations, to discrete, lower-precision values. This reduces the number of bits required to represent these values, thereby limiting the range of representable values.

- **Pros:**
 - Significantly reduces storage requirements and memory usage for the model.
 - Can improve computational efficiency, as operations with fewer bits (especially integers) are generally faster.
- **Cons:**
 - The reduction in precision typically results in some loss of model performance, which needs to be carefully managed.

Numerical Representations

Common Numerical Representations Common numerical representations used in deep learning include:

- **float32**: This is the standard single-precision floating-point format. It uses 1 sign bit, 8 exponent bits, and 23 mantissa bits. Its positive range extends approximately from 1.18×10^{-38} to 3.4×10^{38} .
- **float16**: Also known as half-precision, it uses 1 sign bit, 5 exponent bits, and 10 fraction bits. Its positive range is much smaller, approximately from 6.1×10^{-5} to 6.5×10^4 .
- **bfloat16**: Brain Floating Point, it uses 1 sign bit, 8 exponent bits, and 7 mantissa bits. Crucially, it shares the same dynamic range as **float32** (due to having 8 exponent bits) but

offers lower precision (due to fewer mantissa bits). This makes it robust to large values.

- **int8**: This represents integer values, typically ranging from -128 to 127 .

Quantization Process

Quantization Mapping Scheme The core of **quantization** involves a **mapping scheme** that transforms values from a higher-precision domain (e.g., **float32**) to a lower-precision domain (e.g., **int8**). This scheme is typically defined by a **scale** factor (S) and a **zero-point** (Z).

- The formula for **quantization** is often: $Q = \text{round}(F/S + Z)$, where Q is the quantized integer, and F is the floating-point value. The dequantization formula is $F = (Q - Z) \times S$.
- **Absmax quantization**: This is a **symmetric** scaling method. It identifies the largest absolute value (a) within a tensor and maps $-a$ to the minimum quantized value (e.g., -127 for **int8**) and $+a$ to the maximum quantized value (e.g., $+127$ for **int8**). A key advantage is that it inherently preserves the original 0 value.

Absmax Quantization Example For a **float32** tensor with values from -10 to 10 , and **int8** range $[-127, 127]$, $S = 10/127$. A float value of 5 would be quantized to $\text{round}(5/(10/127)) = \text{round}(63.5) = 64$.

- **Zero-point quantization**: This is an **asymmetric** scaling method. It maps the minimum floating-point value ($\text{min} = a$) in the tensor to the minimum quantized value (e.g., -128 for **int8**) and the maximum floating-point value ($\text{max} = b$) to the maximum quantized value (e.g., $+127$ for **int8**). This approach is generally more efficient for distributions that are not symmetric around zero.

Zero-point Quantization Example For a **float32** tensor with values from 0 to 10 , and **int8** range $[-128, 127]$, $S = (10 - 0)/(127 - (-128)) = 10/255$. $Z = -128 - \text{round}(0/S) = -128$. A float value of 5 would be quantized to $\text{round}(5/(10/255) - 128) = \text{round}(127.5) - 128 = -1$. (Note: precise Z calculation can vary.)

Types of Quantization

- **Post-Training Quantization (PTQ)**: This method quantizes the model weights and/or activations *after* the model has been fully trained in **full precision**. It is straightforward to apply as it does not require any retraining. However, it can sometimes yield sub-optimal performance due to the sudden loss of precision.

- **Quantization-Aware Training (QAT):** This approach incorporates **quantization** directly during the training process.
 - **Forward pass:** During the **Forward pass**, (fake) quantized weights and activations are used. This means the values are rounded to their lower-precision equivalents, but the underlying representation might still be **full precision** for computational stability.
 - **Backward pass:** Gradients are computed in **full precision**. Crucially, **QAT** employs various tricks (e.g., **Straight-Through Estimator**) to handle the non-differentiable rounding operations, allowing the model to learn optimal **quantization** parameters (like **scale** and **zero-point**) alongside the model weights.
 - **QAT** typically offers better performance compared to **PTQ** because the model "learns" to be robust to **quantization** from the start, but it requires intervention in the training loop.

Static vs. Dynamic Quantization (for Activations)

***Activation Quantization Strategies** The **scale** and **zero-point** parameters are essential for quantizing both weights and activations. While these parameters can be precomputed and fixed for weights, activations often require different strategies:*

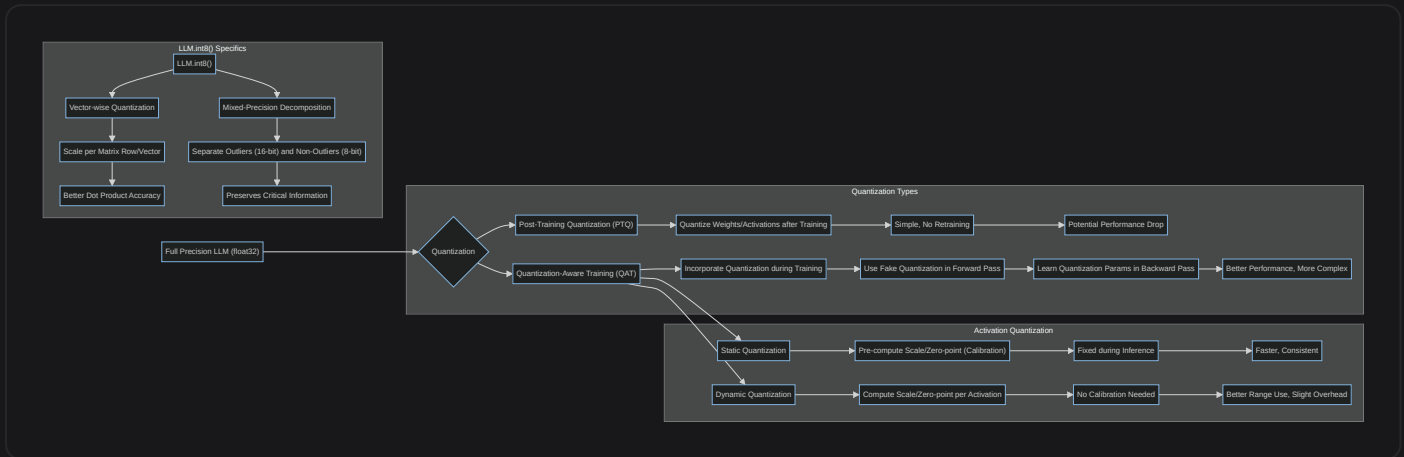
- **Static quantization:** For **Static quantization**, the **scale** and **zero-point** for activations are pre-computed and fixed before inference. This is typically done during a **calibration phase**, where a small representative validation dataset is run through the model, and the ranges of activations are observed to determine optimal **quantization** parameters. This approach leads to faster and more consistent inference.
- **Dynamic quantization:** In **Dynamic quantization**, the **scale** and **zero-point** for activations are computed *on-the-fly* for each activation tensor during inference. This allows for better utilization of the **quantization** range for each specific input, and it eliminates the need for a **calibration phase**. However, this dynamic computation adds a small computational overhead at inference time.

`LLM.int8()`

***LLM.int8()** This is a specialized **quantization** technique designed specifically for **Large Language Models**.*

- **Vector-wise quantization:** Instead of applying a single scaling constant to an entire weight matrix, **LLM.int8()** computes separate scaling constants for each *matrix row or vector*. This fine-grained scaling allows for much better-quantized dot products, especially when dealing with the diverse value ranges found across different parts of an **LLM**'s weights.

- **Mixed-precision decomposition:** A critical feature of `LLM.int8()` is its ability to handle "outlier" weights or activations. These are crucial, unusually large magnitude values (often representing only about 0.1% of features) that can disproportionately impact performance if quantized to `int8`. `LLM.int8()` addresses this by decomposing matrices into two parts: a majority of **8-bit** quantized "non-outlier" values and a small fraction of "outlier" values that are kept in higher precision (e.g., **16-bit**). This **mixed-precision approach** preserves critical information while still achieving significant memory savings.



Reduced Floating Point Precision

Reduced Floating Point Precision Converting a model to **half precision** (`float16` or `bfloat16`) is a less drastic optimization compared to full **quantization**.

- While `float32` (single precision) is the default for many deep learning operations, it has been observed that lower precision is often sufficient for maintaining acceptable model performance.
- This conversion effectively saves half the model's memory footprint, as each parameter now occupies 16 bits instead of 32 bits.
- It is important to note that this is *not* **quantization** in the strict sense, because the values still exist within a continuous range, albeit with fewer bits representing their mantissa and/or exponent.
- In PyTorch, this conversion is straightforward:

```
model = model.half() # for float16
```

or

```
model = model.bfloat16() # for bfloat16 if supported by hardware
```

Model Distillation

Model distillation This technique involves training a smaller, more efficient model (referred to as the **student** model) to mimic the behavior and knowledge of a larger, more complex, and typically higher-performing model (the **teacher** model). The goal is to "distill" the essential knowledge from the **teacher** into the **student**.

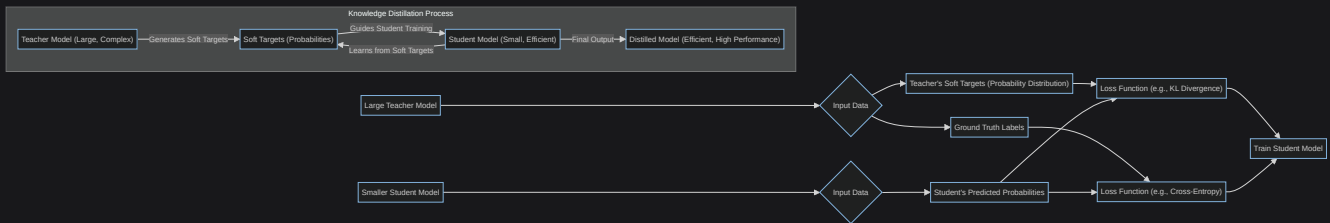
- Instead of merely training the **student** model on the ground truth labels, the **student** is trained to predict the **teacher's probability distribution** over all possible output classes (e.g., across all words in a vocabulary) for a given input.
- Learning from the **teacher's "soft targets"** (the **full probability distribution**) provides much richer and more nuanced semantic information than simply learning from "**hard**" ground truth labels.

Soft Targets in Model Distillation For the input "The cat sat on the mat", if the ground truth label for a token is "cat", a **hard target** would be (cat : 1, everythingelse : 0). However, a **teacher** model might provide a **soft target** like (cat : 0.6, kitten : 0.2, kitty : 0.1, cats : 0.1, dog : 0.001, etc.). This **soft distribution** tells the **student** not only that "cat" is the correct answer but also that "kitten" and "kitty" are semantically very similar and plausible alternatives, while "dog" is highly unlikely. This extra information helps the **student** learn better generalizations.

Model Distillation in LLMs

Distilled LLM Examples Distilled models, despite their smaller architectures, have consistently achieved performance levels comparable to their much larger **teacher** counterparts.

- **DistilBERT** (distilled from **BERT**): This **student** model is 40% smaller than **BERT**, retains approximately 97% of **BERT's** performance, and offers a 60% faster inference speed.
- **TinyBERT** (also distilled from **BERT**): This model is even more compact, being 7.5x smaller than **BERT**. It still manages to retain about 97% of **BERT's** performance while achieving a remarkable 9.4x faster inference.
- **MiniLM**: Another example, this model is 50% smaller and maintains 99% performance retention relative to its **teacher**.
- **DistilGPT-2**: A distilled version of **GPT-2**, showcasing the applicability of **distillation** to generative language models.



References

- Large Language Models (LLMs)
- Fine-tuning LLMs
- Parameter-Efficient Fine-Tuning (PEFT)
- Low-Rank Adaptation (LoRA)
- Quantization
- Model Distillation
- BERT
- GPT-3
- FLAN
- Instruction Tuning
- Model Alignment
- Adapter Layers
- Prompt Tuning
- BitFit
- Numerical Representations
- Absmax Quantization
- Zero-Point Quantization
- Post-Training Quantization (PTQ)
- Quantization-Aware Training (QAT)
- Straight-Through Estimator
- Static Quantization
- Dynamic Quantization
- LLM.int8()
- DistilBERT
- TinyBERT
- MiniLM

